



PUESTA EN PRODUCCIÓN
SEGURA

BROKEN AUTHENTICATION

Carles Ochoa

Índice

¿Qué es Broken Authentication?.....	3
Qué realizaremos en esta práctica?.....	3
Creación de la base de datos.....	3
Explotación de Broken Authentication.....	7
Mitigación: Código Seguro en PHP.....	8
Implementar bloqueo de cuenta tras varios intentos fallidos.....	13
Implementar Autenticación Multifactor (MFA).....	15
Conclusión.....	19

¿Qué es Broken Authentication?

Broken Authentication es cuando el sistema tiene errores o malas prácticas en el código de autenticación que permiten a un atacante iniciar sesión sin mucha dificultad, ya sea adivinando, saltándose o manipulando el proceso de login.

La autenticación está “rota” porque fue implementada de forma débil, insegura o incorrecta.

Qué realizaremos en esta práctica?

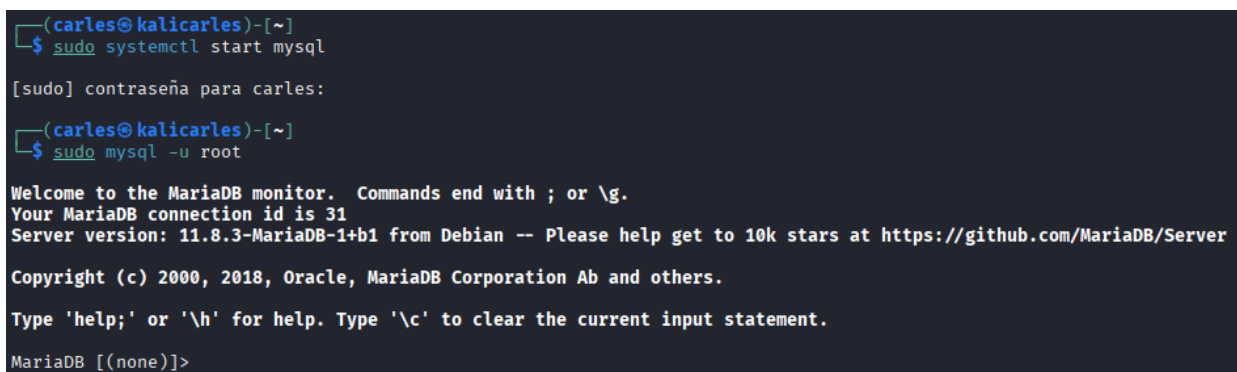
En esta práctica vamos a aprender cómo funciona una autenticación débil y cómo puede ser explotada por un atacante para obtener acceso no autorizado a un sistema. Para ello, primero montaremos un entorno vulnerable y realizaremos ataques reales, como fuerza bruta e inyección SQL. Después aplicaremos las medidas necesarias para corregir las vulnerabilidades y fortalecer el proceso de autenticación, incluyendo hashing seguro de contraseñas, consultas preparadas, bloqueo por intentos fallidos y autenticación multifactor (MFA).

Para ello seguiremos esta serie de pasos:

Creación de la base de datos

El primer paso para la creación de la base de datos es iniciar el servicio y entrar como root:

- `sudo systemctl start mysql`
- `sudo mysql -u root`



```
(carles@kalicarles)-[~]
$ sudo systemctl start mysql
[sudo] contraseña para carles:
(carles@kalicarles)-[~]
$ sudo mysql -u root

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.8.3-MariaDB-1+b1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Una vez iniciada la sesión ejecutaremos el comando para la creación de la base de datos y la creación de la tabla usuarios:

- `CREATE DATABASE testdb;`
- `USE testdb;`

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  password VARCHAR(50) NOT NULL
```

);

Con la tabla de usuarios ya creada insertaremos un usuario para realizar las pruebas:

- `INSERT INTO users (username, password) VALUES ('admin', '123456');`

Con el comando `SELECT * FROM users;` podemos verificar que todo está creado.

Y por último ejecutamos `SET PASSWORD FOR 'root'@'localhost' = PASSWORD('root');` para insertar una contraseña al root.

```
MariaDB [(none)]> CREATE DATABASE testdb;
Query OK, 1 row affected (0,001 sec)

MariaDB [(none)]> USE testdb;
Database changed
MariaDB [testdb]> CREATE TABLE users (
  → id INT AUTO_INCREMENT PRIMARY KEY,
  → username VARCHAR(50) UNIQUE NOT NULL,
  → password VARCHAR(50) NOT NULL
  → );
Query OK, 0 rows affected (0,098 sec)

MariaDB [testdb]> INSERT INTO users (username, password) VALUES ('admin', '123456');
Query OK, 1 row affected (0,015 sec)

MariaDB [testdb]> SELECT * FROM users;
+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | admin   | 123456   |
+----+-----+-----+
1 row in set (0,001 sec)

MariaDB [testdb]> exit;
Bye

(carles@kalicarles)~
```

Código Vulnerable: login_weak.php

El siguiente paso es crear el archivo vulnerable login_weak.php.. El código contiene varias vulnerabilidades que pueden ser explotadas para realizar ataques de autenticación rota.

Para ello crearemos el archivo dentro de la carpeta del apache:

- `cd /var/www/html/`
- `sudo nano login_weak.php`
- Insertamos el código

```
<?php
```

```
$conn = new mysqli("localhost", "root", "root", "testdb");
```

```
if ($conn->connect_error) {
```

```
    die("Error de conexión: " . $conn->connect_error);
```

```
}
```

```
if ($_SERVER["REQUEST_METHOD"] == "POST" || $_SERVER["REQUEST_METHOD"] ==
"GET") {
    $username = $_REQUEST["username"];
    $password = $_REQUEST["password"];
    print("Usuario: " . $username . "<br>");
    print("Contraseña: " . $password . "<br>");
    $query = "SELECT * FROM users WHERE username = '$username' AND password =
'$password'";
    print("Consulta SQL: " . $query . "<br>");
    $result = $conn->query($query);
    if ($result->num_rows > 0) {
        echo "Inicio de sesión exitoso";
    } else {
        echo "Usuario o contraseña incorrectos";
    }
}
?>
```

- Reiniciamos el servicio: `sudo systemctl restart apache2`

```
(carles@kalicarles)-[~]
$ sudo service apache2 start

(carles@kalicarles)-[~]
$ cd /var/www/html/

(carles@kalicarles)-[/var/www/html]
$ sudo nano login_weak.php

(carles@kalicarles)-[/var/www/html]
$ cat login_weak.php
<?php
$conn = new mysqli("localhost", "root", "root", "testdb");
if ($conn->connect_error) {
    die("Error de conexión: " . $conn->connect_error);
}
if ($_SERVER["REQUEST_METHOD"] == "POST" || $_SERVER["REQUEST_METHOD"] == "GET") {
    $username = $_REQUEST["username"];
    $password = $_REQUEST["password"];
    print("Usuario: " . $username . "<br>");
    print("Contraseña: " . $password . "<br>");
    $query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
    print("Consulta SQL: " . $query . "<br>");
    $result = $conn->query($query);

    if ($result->num_rows > 0) {
        echo "Inicio de sesión exitoso";
    } else {
        echo "Usuario o contraseña incorrectos";
    }
}
?>

(carles@kalicarles)-[/var/www/html]
$ sudo service apache2 restart

(carles@kalicarles)-[/var/www/html]
$
```

Explotación de Broken Authentication

1. Ataque de fuerza bruta con Hydra

El primer paso será descomprimir el archivo rockyou.txt.gz, que es un diccionario de contraseñas muy usado en ataques de fuerza bruta; para ello ejecutaremos el comando: `sudo gunzip /usr/share/wordlists/rockyou.txt.gz`

Con el fichero ya descomprimido ejecutaremos en la terminal el siguiente comando:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost http-post-form  
"/login_weak.php:username=^USER^&password=^PASS^:Usuario o contraseña incorrectos" -V
```

- `-l admin` → usuario al que queremos atacar
- `-P rockyou.txt` → diccionario de contraseñas
- `localhost` → atacas a nuestro propio servidor
- `http-post-form` → tipo de ataque
- Dentro de las comillas
 - `/login_weak.php` → ruta del login vulnerable
 - `username=^USER^&password=^PASS^` → Hydra sustituye estos valores
 - `Usuario o contraseña incorrectos` → mensaje que indica un login fallido

Si ha funcionado, hydra nos mostrará la contraseña y en este caso si que ha funcionado por lo tanto podemos confirmar con hydra:

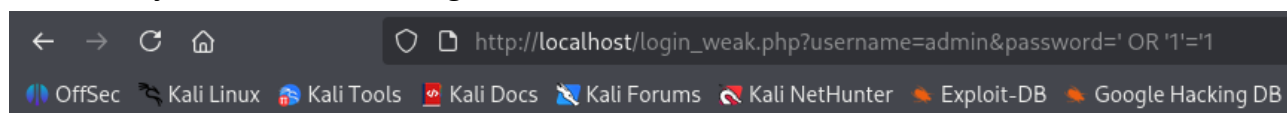
- el login es vulnerable
- no existe ningún límite de intentos
- la autenticación está “rota”

2. Explotación de SQL Injection

Mediante un payload malicioso en el campo de la contraseña veremos como lograr iniciar la sesión correctamente.

La url ha introducir es `http://localhost/login_weak.php?username=admin&password=' OR '1'='1`.

En este caso como `'1'='1` siempre es verdadero el sistema nos dejará entrar aunque no sepamos la contraseña y el resultado será el siguiente:



Usuario: admin

Contraseña: ' OR '1'='1

Consulta SQL: `SELECT * FROM users WHERE username = 'admin' AND password = ' OR '1'='1'`

Inicio de sesión exitoso

Mitigación: Código Seguro en PHP

En esta fase vamos a corregir todas las debilidades presentes en el login vulnerable. El sistema original almacenaba las contraseñas en texto plano, permitía realizar SQL Injection y no validaba correctamente los datos recibidos. Ahora implementaremos las prácticas adecuadas para transformar ese login inseguro en un login correcto y seguro.

1. Modificar la tabla para soportar contraseñas seguras (hashing)

Las contraseñas ya no podrán guardarse en texto plano. Usaremos `password_hash()`, que genera un hash de hasta 60 caracteres. Por lo tanto en mysql ejecutaremos:

```
ALTER TABLE users MODIFY password VARCHAR(255) NOT NULL;
```

Esto permite almacenar cualquier hash generado con los algoritmos recomendados en PHP.

```
(carles@kalicarles)-[~]
$ sudo mysql -u root -p

[sudo] contraseña para carles:
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 77
Server version: 11.8.3-MariaDB-1+b1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE testdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [testdb]> ALTER TABLE users MODIFY password VARCHAR(255) NOT NULL;
Query OK, 1 row affected (0.198 sec)
Records: 1 Duplicates: 0 Warnings: 0

MariaDB [testdb]> █
```

2. Insertar usuarios con password hashing

Creamos un archivo PHP que insertará un usuario seguro, ya con contraseña hasheada.

Creamos el archivo con `sudo nano add_users.php` y insertamos el código.


```
(carles@kalicarles)-[/var/www/html]
$ cat add_users.php
<?php
error_reporting(E_ALL);
ini_set('display_errors', 1);

$conn = new mysqli("localhost", "root", "root", "testdb");
if ($conn->connect_error) {
    die("Conexión fallida: " . $conn->connect_error);
}

$username = "raul";
$password = "123456";

$hashed_password = password_hash($password, PASSWORD_DEFAULT);

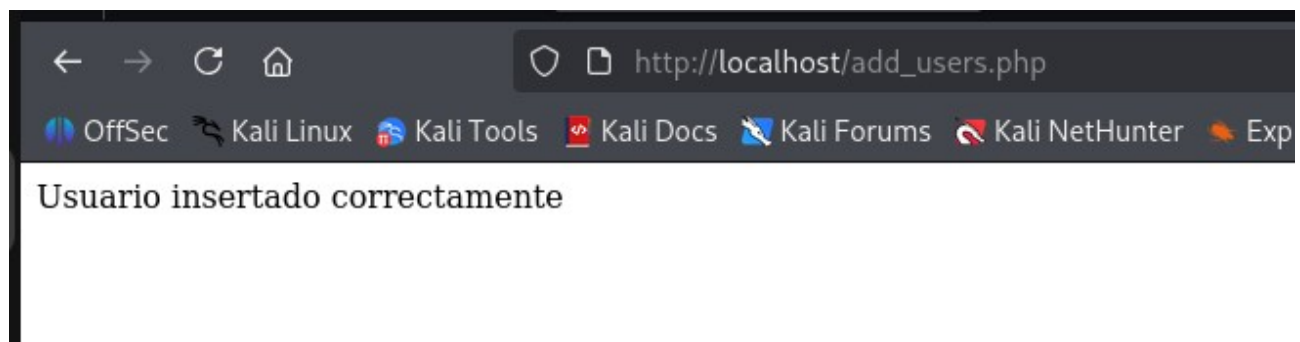
$stmt = $conn->prepare("INSERT INTO users (username, password) VALUES (?, ?)");
$stmt->bind_param("ss", $username, $hashed_password);

if ($stmt->execute()) {
    echo "Usuario insertado correctamente";
} else {
    echo "Error al insertar usuario: " . $stmt->error;
}

$stmt->close();
$conn->close();
?>

(carles@kalicarles)-[/var/www/html]
$
```

Si ahora ejecutamos esta página en el navegador la salida debe ser que el usuario se ha insertado correctamente



Y en la tabla users de la base de datos veremos que el usuario nuevo ya tiene la contraseña hasheada.

```
ers at line 1
MariaDB [testdb]> select * from users;
+-----+-----+-----+
| id | username | password |
+-----+-----+-----+
| 1 | admin | 123456 |
| 2 | raul | $2y$12$X376RuzVHP5oPUPm0WHR.kQ6nKvHic3.ZcOf1lK8qn/kL0etDwJi |
+-----+-----+-----+
2 rows in set (0,001 sec)

MariaDB [testdb]> █
```

3. Crear login seguro con password_verify()

Ahora modificamos el login para que ya no compare contraseñas directamente, sino que use la función segura password_verify().

Creamos un nuevo archivo con sudo nano login_secure.php y insertamos el código.

```
(carles@kalicarles)-[/var/www/html]
$ cat login_secure.php
<?php
$conn = new mysqli("localhost", "root", "root", "testdb");

if ($conn->connect_error) {
    die("Error de conexión: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST" || $_SERVER["REQUEST_METHOD"] == "GET") {

    $username = $_REQUEST["username"];
    $password = $_REQUEST["password"];

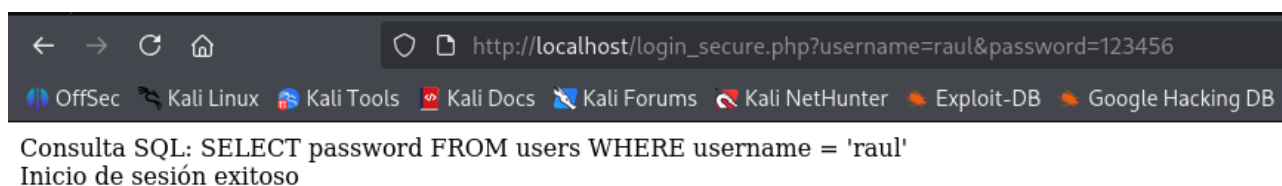
    $query = "SELECT password FROM users WHERE username = '$username'";
    print("Consulta SQL: " . $query . "<br>");

    $result = $conn->query($query);

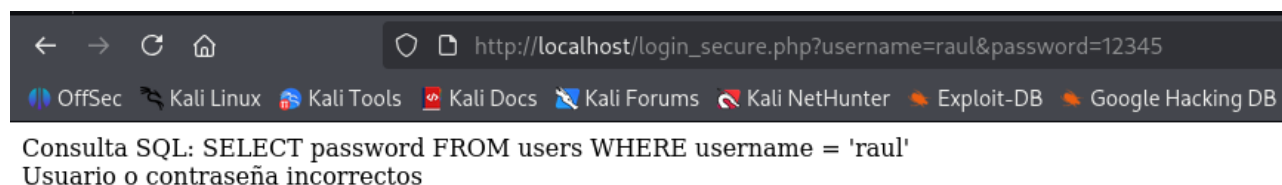
    if ($result->num_rows > 0) {
        $row = $result->fetch_assoc();
        $hashed_password = $row["password"];

        if (password_verify($password, $hashed_password)) {
            echo "Inicio de sesión exitoso";
        } else {
            echo "Usuario o contraseña incorrectos";
        }
    } else {
        echo "Usuario no encontrado";
    }
}
```

Si ahora insertamos la url http://localhost/login_secure.php?username=raul&password=123456 en el navegador, la sesión estará iniciada porque sabemos que esa es la contraseña del usuario raul.



Pero si la introducimos mal veremos que nos aparece que el usuario o contraseña son incorrectos.



4. Prevenir SQL Injection con consultas preparadas

El login vulnerable permitía SQL Injection porque insertaba directamente los datos del usuario dentro de la consulta. Para evitarlo usamos consultas preparadas, donde la SQL se define primero y los valores se envían por separado. Esto hace que cualquier texto introducido por el usuario se trate solo como dato y nunca como parte del código SQL, eliminando así la posibilidad de alterar la consulta y bloqueando la inyección. **Además, este nuevo código seguro reemplaza por completo a los anteriores, ya que engloba todas las mejoras necesarias para impedir que el atacante pueda manipular la autenticación.**

Para ello en un nuevo archivo php sudo nano login_prepared.php introduciremos este código:

```
(carles@kalicarles)-[/var/www/html]
$ cat login_prepared.php
<?php
$conn = new mysqli("localhost", "root", "root", "testdb");

if ($conn->connect_error) {
    die("Error de conexión: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST" || $_SERVER["REQUEST_METHOD"] == "GET") {

    $username = $_REQUEST["username"];
    $password = $_REQUEST["password"];

    $query = "SELECT password FROM users WHERE username = ?";
    $stmt = $conn->prepare($query);
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $stmt->store_result();

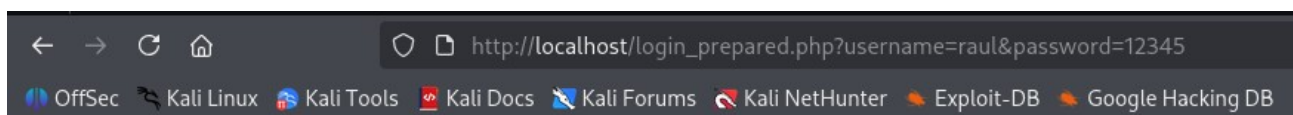
    if ($stmt->num_rows > 0) {
        $stmt->bind_result($hashed_password);
        $stmt->fetch();

        if (password_verify($password, $hashed_password)) {
            echo "Inicio de sesión exitoso";
        } else {
            echo "Usuario o contraseña incorrectos";
        }
    } else {
        echo "Usuario no encontrado";
    }
}
```

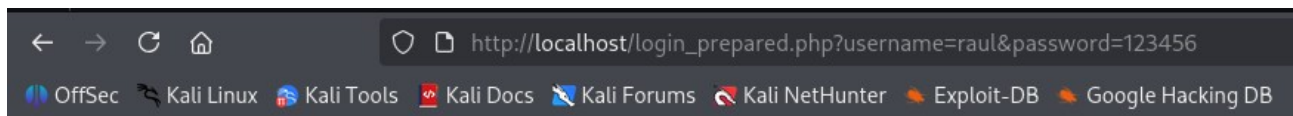
Este login ya:

- No permite SQL Injection
- Usa hashing seguro
- No expone información sensible
- Valida correctamente los datos

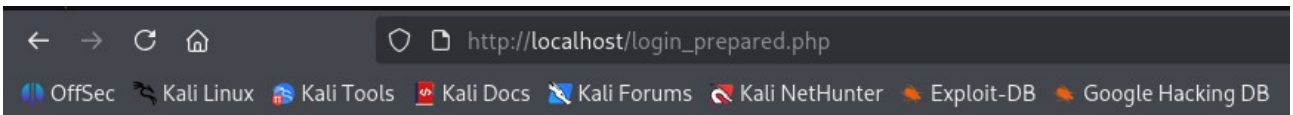
Y si ejecutamos diversas pruebas, los resultados serán los siguientes:



Usuario o contraseña incorrectos



Inicio de sesión exitoso



Usuario no encontrado

Implementar bloqueo de cuenta tras varios intentos fallidos

Esta es la siguiente mejora de seguridad y forma parte de la mitigación de Broken Authentication.

Sirve para evitar ataques de fuerza bruta como el que hicimos con Hydra.

Por lo tanto vamos a implementar un sistema donde:

- Si un usuario falla 3 veces la contraseña → la cuenta se bloquea temporalmente
- Se registra el momento del último intento
- Pasados 15 minutos, los intentos fallidos se reinician
- Si inicia sesión correctamente → los intentos fallidos vuelven a 0

Esto evita que Hydra u otro atacante pueda probar miles de contraseñas seguidas.

Por lo tanto en la tabla users de la base de datos vamos a añadir dos columnas nuevas:

- failed_attempts → número de intentos fallidos
- last_attempt → fecha y hora del último intento

```
ALTER TABLE users ADD failed_attempts INT DEFAULT 0;
```

```
ALTER TABLE users ADD last_attempt TIMESTAMP NULL DEFAULT NULL;
```

```
(carles@kalicarles)-[/var/www/html]
$ sudo mysql -u root -p

[sudo] contraseña para carles:
Lo siento, pruebe otra vez.
[sudo] contraseña para carles:
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 36
Server version: 11.8.3-MariaDB-1+b1 from Debian -- Please help get to 10k stars
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> use testdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [testdb]> ALTER TABLE users ADD failed_attempts INT DEFAULT 0;
Query OK, 0 rows affected (0,119 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [testdb]> ALTER TABLE users ADD last_attempt TIMESTAMP NULL DEFAULT NULL;
Query OK, 0 rows affected (0,123 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [testdb]> █
```

Ahora creamos el archivo php para insertar el código

```
(carles@kalicarles)-[/var/www/html]
$ cat login_3intentos.php
<?php
$conn = new mysqli("localhost", "root", "root", "testdb");

if ($conn->connect_error) {
    die("Error de conexión: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST" || $_SERVER["REQUEST_METHOD"] == "GET") {

    $username = $_REQUEST["username"];
    $password = $_REQUEST["password"];

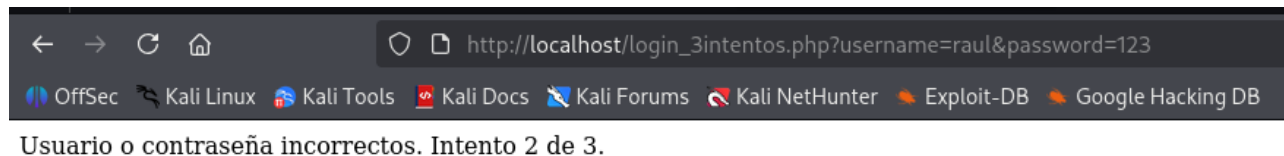
    $query = "SELECT password, failed_attempts, last_attempt FROM users WHERE username = ?";
    $stmt = $conn->prepare($query);
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $stmt->store_result();

    if ($stmt->num_rows > 0) {
        $stmt->bind_result($hashed_password, $failed_attempts, $last_attempt);
        $stmt->fetch();
        $stmt->close();

        $current_time = time();
        $lockout_duration = 900; // 15 minutos

        if ($failed_attempts ≥ 3) {
            $last_attempt_time = strtotime($last_attempt);
```

Y desde el navegador ejecutamos las pruebas. Si fallamos la contraseña vemos que se incrementa el número de errores



Y en la columna `failed_attempts` vemos el número de intentos que lleva el usuario raul

```
(carles@kalicarles)-[/var/www/html]
$ sudo mysql -u root -p

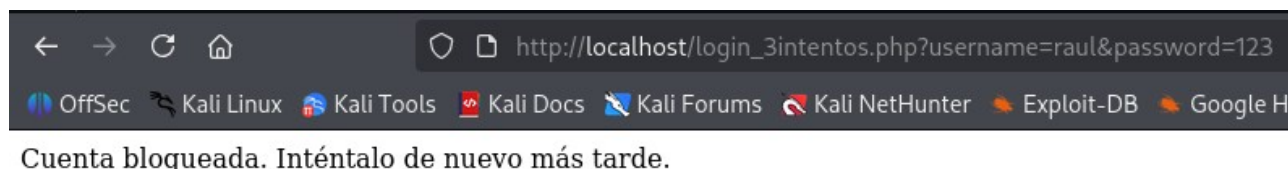
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 33
Server version: 11.8.3-MariaDB-1+b1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> use testdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [testdb]> select * from users;
+----+-----+-----+-----+-----+
| id | username | password | failed_attempts | last_attempt |
+----+-----+-----+-----+-----+
| 1  | admin   | 123456   | 0              | NULL         |
| 2  | raul    | $2y$12$X376RuzVHP5oPUPm0WHr.kQ6nKvHic3.ZcOf1lK8qn/kL0etDwJi | 2              | 2025-12-04 22:37:31 |
+----+-----+-----+-----+-----+
2 rows in set (0,001 sec)
```

Si fallamos los 3 intentos este es el resultado (tendríamos que esperar 15 minutos):



Implementar Autenticación Multifactor (MFA)

Este paso añade una segunda capa de seguridad: el usuario ya no puede entrar solo con usuario y contraseña; además necesita un código OTP temporal enviado por “correo” (simulado en un archivo .txt). Esto evita que un atacante pueda acceder incluso si consigue la contraseña, ya que deberá ingresar también el código de verificación generado en cada inicio de sesión.

1. Modificación en la Base de Datos

El primer paso es modificar la Base de Datos para agregar una nueva columna para almacenar el OTP temporal y su tiempo de expiración:

```
Database changed
MariaDB [testdb]> ALTER TABLE users ADD otp_code VARCHAR(6) NULL;
Query OK, 0 rows affected (0,167 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [testdb]> ALTER TABLE users ADD otp_expires TIMESTAMP NULL DEFAULT NULL;
Query OK, 0 rows affected (0,123 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [testdb]> ALTER TABLE users ADD email VARCHAR(255) NULL;
Query OK, 0 rows affected (0,118 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [testdb]> select * from users;
+----+-----+-----+-----+-----+-----+-----+
| id | username | password | failed_attempts | last_attempt | otp_code | otp_expires | email |
+----+-----+-----+-----+-----+-----+-----+
| 1 | admin | 123456 | 0 | NULL | NULL | NULL | NULL |
| 2 | raul | $2y$12$X376RuzVHP5oPUPm0WHr.kQ6nKvHic3.Zc0f1lK8qn/kL0etDwJi | 3 | 2025-12-04 22:39:19 | NULL | NULL | NULL |
+----+-----+-----+-----+-----+-----+-----+
2 rows in set (0,002 sec)

MariaDB [testdb]> bye
→ ^C
MariaDB [testdb]> exit
Bye

(carles@kalicarles)-[~]
```

El siguiente paso es crear el archivo del login con MFA: `sudo nano login_otp.php`

```
(carles@kalicarles)-[/var/www/html]
$ cat login_otp.php
<?php
$conn = new mysqli("localhost", "root", "root", "testdb");

if ($conn->connect_error) {
    die("Error de conexión: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST" || $_SERVER["REQUEST_METHOD"] == "GET") {

    $username = $_REQUEST["username"];
    $password = $_REQUEST["password"];

    $query = "SELECT id, password, email, failed_attempts, last_attempt
              FROM users WHERE username = ?";
    $stmt = $conn->prepare($query);
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $stmt->store_result();

    if ($stmt->num_rows > 0) {

        $stmt->bind_result($user_id, $hashed_password, $email, $failed_attempts, $last_attempt);
        $stmt->fetch();
        $stmt->close();

        if (password_verify($password, $hashed_password)) {

            $otp = mt_rand(100000, 999999);
            $otp_expires = date('Y-m-d H:i:s', strtotime('+10 minutes'));

            $update_otp = "UPDATE users SET otp_code = ?, otp_expires = ? WHERE id = ?";
            $stmt = $conn->prepare($update_otp);
```

2. Creación del archivo donde el usuario introduce el OTP

Creamos el fichero `verificar_otp.php` y insertamos el código:


```
(carles@kalicarles)-[/var/www/html]
$ cat verificar_otp.php
<?php
session_start();

$conn = new mysqli("localhost", "root", "root", "testdb");

if ($conn->connect_error) {
    die("Error de conexión: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {

    $user_id = $_SESSION['user_id'];
    $otp_ingresado = $_POST["otp"];

    $query = "SELECT otp_code, otp_expires FROM users WHERE id = ?";
    $stmt = $conn->prepare($query);
    $stmt->bind_param("i", $user_id);
    $stmt->execute();
    $stmt->bind_result($otp_almacenado, $otp_expires);
    $stmt->fetch();
    $stmt->close();

    if ($otp_almacenado == $otp_ingresado && strtotime($otp_expires) > time()) {
        echo "Autenticación exitosa. Bienvenido!";
    } else {
        echo "Código OTP inválido o expirado.";
    }
}

$conn->close();
?>
```

3. Crear el archivo donde se “simula” el envío del correo

En este paso crearemos el fichero emails_simulados.txt que almacenará el código OTP simulando el envío del correo. Para su creación los comandos son:

- `cd /var/www/html`
- `sudo touch emails_simulados.txt`
- `sudo chmod 777 emails_simulados.txt`

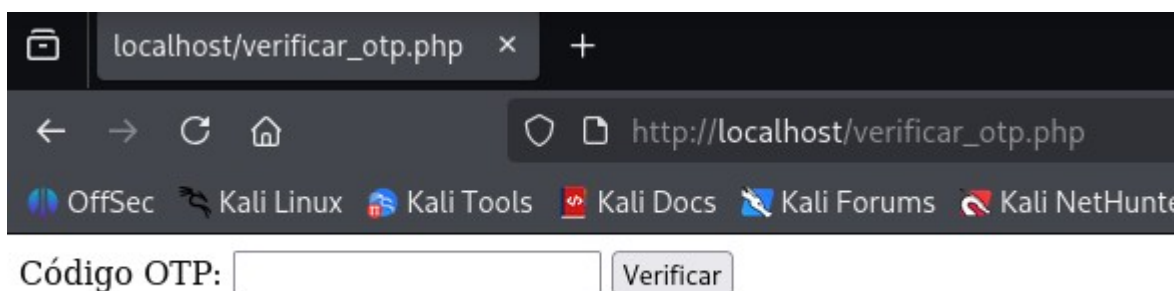
```
(carles@kalicarles)-[/var/www/html]
$ sudo touch emails_simulados.txt

(carles@kalicarles)-[/var/www/html]
$ sudo chmod 777 emails_simulados.txt

(carles@kalicarles)-[/var/www/html]
$
```

Con el fichero ya creado insertaremos la url

http://localhost/login_otp.php?username=raul&password=123456 en el navegador para que el OTP del usuario raul se almacene. Y seremos redirigidos automáticamente a la url http://localhost/verificar_otp.php

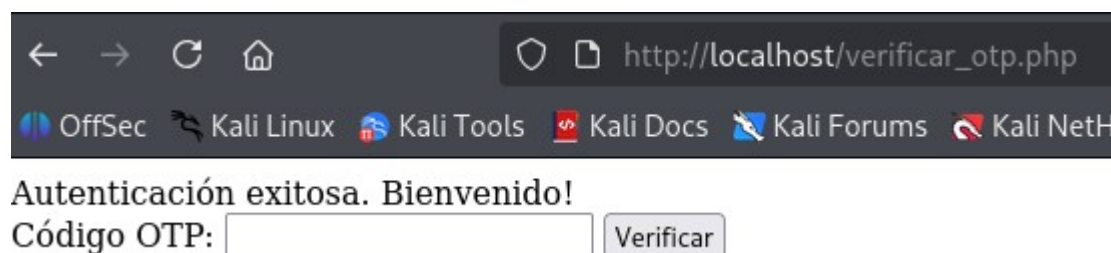


Ahora en el fichero emails_simulados.txt aparecerá el código OTP.

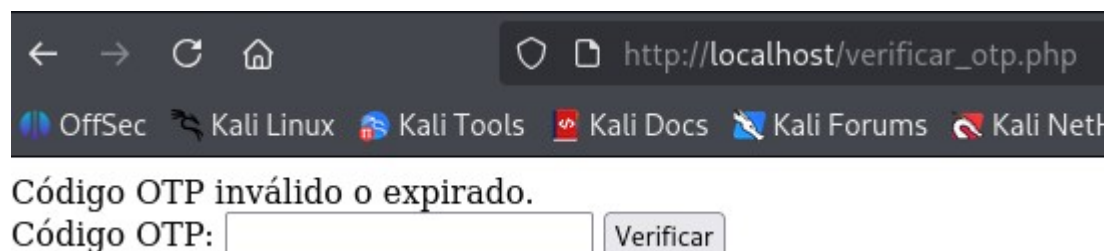
```
(carles@kalicarles)-[/var/www/html]
$ cat emails_simulados.txt
[2025-12-05 08:39:56] To:
OTP: 616958

(carles@kalicarles)-[/var/www/html]
$
```

Si lo introducimos en la página de verificar_otp.php veremos que nos aparece un mensaje informándonos que la autenticación ha sido exitosa.



Y si introducimos mal el código OTP la salida será la siguiente:



En esta parte de la práctica implementamos un sistema de autenticación multifactor (MFA) basado en un código OTP temporal. Esto significa que el usuario ya no puede acceder solo con su nombre de usuario y su contraseña, sino que después debe introducir un segundo código que cambia en cada inicio de sesión. En un entorno real este código se enviaría por correo electrónico, pero aquí lo simulamos escribiéndolo en un archivo llamado emails_simulados.txt, que actúa como si fuera la bandeja de entrada del usuario.

El encargado de generar el OTP y “enviarlo” a este archivo es el propio fichero login_otp.php, concretamente la función enviarCorreo(). Esta función recibe el correo del usuario y el código OTP generado, y en lugar de enviarlo por un servidor de correo real, lo guarda dentro del archivo emails_simulados.txt. De esta forma podemos ver el OTP generado, comprobar su validez y completar la autenticación multifactor sin necesidad de un sistema de correo real.

Conclusión

Esta práctica ha permitido comprender de manera completa cómo una autenticación débil puede comprometer por completo la seguridad de un sistema. A través del entorno vulnerable, hemos demostrado que fallos como contraseñas en texto plano, ausencia de limitación de intentos y consultas SQL inseguras permiten a un atacante obtener acceso sin dificultad mediante fuerza bruta o SQL Injection.

Tras la fase de explotación, hemos aplicado diferentes medidas de mitigación que reflejan las buenas prácticas reales en seguridad: uso de contraseñas hasheadas, consultas preparadas, bloqueo de cuenta y autenticación multifactor. Gracias a estas mejoras, el sistema pasó de ser fácilmente vulnerable a ofrecer un nivel de protección sólido frente a ataques comunes.